

DAFTAR ISI

BAB I PENYAJIAN MATERI.....	2
1.1. Ekosistem Perangkat Lunak untuk Data Science dan Machine Learning	2
1.1.1. Python	2
1.1.2. Komponen Ekosistem.....	3
1.2. Menyiapkan Lingkungan Kerja.....	3
1.3. Dasar-dasar Python untuk Analisis Data	3
1.4. Pandas: DataFrame dan Series sebagai Struktur Data Utama.....	4
1.4.1 Pengertian DataFrame dan Series	4
1.4.2 Membangun DataFrame dari Awal	4
1.5. Matplotlib dan Seaborn: Pengantar Awal Visualisasi	5
1.6. Memuat dan Memahami Dataset	6
1.7. Inspeksi Data Mendalam	7
1.7.1 Informasi Struktur Dataset dengan df.info()	7
1.7.2 Statistik Deskriptif dengan df.describe().....	8
1.7.3 Mengecek dan Mengukur Missing Value	8
1.8. Penanganan Missing Value Sederhana	9
1.8.1 Menghapus Baris atau Kolom yang Mengandung Missing Value	9
1.8.2 Mengisi Missing Value melalui Imputasi.....	9
1.9. Visualisasi Data Dasar untuk Eksplorasi	10
1.9.1 Histogram: Memvisualisasikan Distribusi Data	10
1.9.2 Boxplot: Ringkasan Lima Serangkai dan Deteksi Outlier	11
1.9.3 Scatter Plot: Mengungkap Hubungan Antar-Variabel	12
1.9.4 Heatmap Korelasi: Mengukur Kekuatan Hubungan Antar-Variabel Secara Kuantitatif.....	12

BAB I

PENYAJIAN MATERI

1.1. Ekosistem Perangkat Lunak untuk Data Science dan Machine Learning

1.1.1. Python

Python merupakan bahasa pemrograman tingkat tinggi dan general-purpose, sehingga dapat digunakan untuk berbagai kebutuhan seperti pengembangan aplikasi, otomasi, komputasi ilmiah, *Data Science*, *Artificial Intelligence*, dan *Machine Learning*. Python populer dalam Data Science karena memiliki sintaks yang relatif mudah dibaca dan ekosistem library yang sangat besar.

Alasan python banyak digunakan

1. Library lengkap

- Numpy (Komputasi Numerik)
- Pandas (manipulasi data)
- Matplotlib (Visualisasi)
- Seaborn (Visualisasi Statistik)
- Scikit-learn (Machine Learning)
- TensorFlow/PyTorch (Deep Learning)

2. *Open Source* dan lengkap

3. Komunitas besar

- Banyak Dokumentasi
- Tutorial
- Forum
- Contoh kode

4. Interaktif

Python dapat digunakan bersama Jupyter Notebook sehingga kode dapat dijalankan sedikit demi sedikit dan hasilnya langsung diamati

1.1.2. Komponen Ekosistem

Ekosistem Python untuk *data science* dapat dibayangkan sebagai sebuah rangkaian alat kerja (*toolchain*) yang saling melengkapi, di mana setiap alat memiliki peran spesifik dalam siklus kerja analisis data. Empat pustaka inti yang akan digunakan secara intensif pada mata kuliah ini adalah sebagai berikut.

1. NumPy (Numerical Python)

Pustaka fundamental yang digunakan untuk komputasi numerik di python

2. Pandas

Pustaka yang digunakan untuk memuat, memeriksa, membersihkan dan mentransformasi data

3. Matplotlib

Pustaka yang digunakan untuk visualisasi data tingkat dasar seperti grafik

4. Seaborn

Pustaka yang digunakan untuk visualisasi data tingkat tinggi seperti *boxplot*, *heatmap*, dan *pairplot*.

NumPy menjadi fondasi komputasi numerik, pandas digunakan untuk mengolah data tabular, sedangkan Matplotlib dan Seaborn digunakan untuk memvisualisasikan data.

1.2. Menyiapkan Lingkungan Kerja

Pada tahap ini terdapat dua opsi dalam menyiapkan lingkungan kerja sebagai tempat kode untuk ditulis. Kita dapat menggunakan instalasi lokal menggunakan anaconda dan jupyter Notebook atau menggunakan Google Collab. Library yang akan digunakan pada praktek kali ini adalah pandas, numpy, matplotlib dan seaborn.

1.3. Dasar-dasar Python untuk Analisis Data

1.3.1 List dan Dictionary

List adalah struktur data bawaan Python yang menampung kumpulan nilai secara terurut (ordered) dan dapat diubah (mutable) setelah dibuat. Sebuah list dituliskan dengan mengapit nilai-nilai di antara tanda kurung siku, dipisahkan tanda koma, misalnya [1, 2, 3] untuk list berisi tiga bilangan bulat, atau ['Pagi', 'Siang', 'Malam']

Dictionary adalah struktur data yang menyimpan data dalam bentuk pasangan kunci-nilai (key-value pairs), dituliskan dengan mengapit pasangan tersebut di antara tanda kurung kurawal, misalnya {'nama': 'Budi', 'usia': 21}.

1.3.2 Array NumPy

List Python kurang efisien untuk operasi matematika dalam jumlah besar. NumPy menyediakan array multidimensi yang dirancang untuk operasi numerik. Contoh penggunaan NumPy

```
import numpy as np  
  
arr = np.array([1, 2, 3, 4])  
  
arr.mean()
```

Kode di atas akan langsung menghasilkan mean dari array

1.4. Pandas: DataFrame dan Series sebagai Struktur Data Utama

1.4.1 Pengertian DataFrame dan Series

DataFrame adalah struktur data dua dimensi berbentuk tabel yang terdiri dari baris dan kolom. DataFrame disajikan persis seperti pada data yang ada di tabel Excel ataupun tabel database. Sedangkan, **Series** adalah struktur data satu dimensi, biasanya merepresentasikan satu kolom DataFrame.

Contoh

```
df['Quantity']
```

1.4.2 Membangun DataFrame dari Awal

Salah satu cara paling sederhana untuk memahami struktur DataFrame adalah dengan membangunnya secara manual dari sebuah dictionary Python, sebagaimana ditunjukkan pada cuplikan kode berikut

```
import pandas as pd  
  
data = {'Nama': ['Ali', 'Budi'], 'Nilai': [85, 90]}  
  
df = pd.DataFrame(data)  
  
print(df)
```

Output

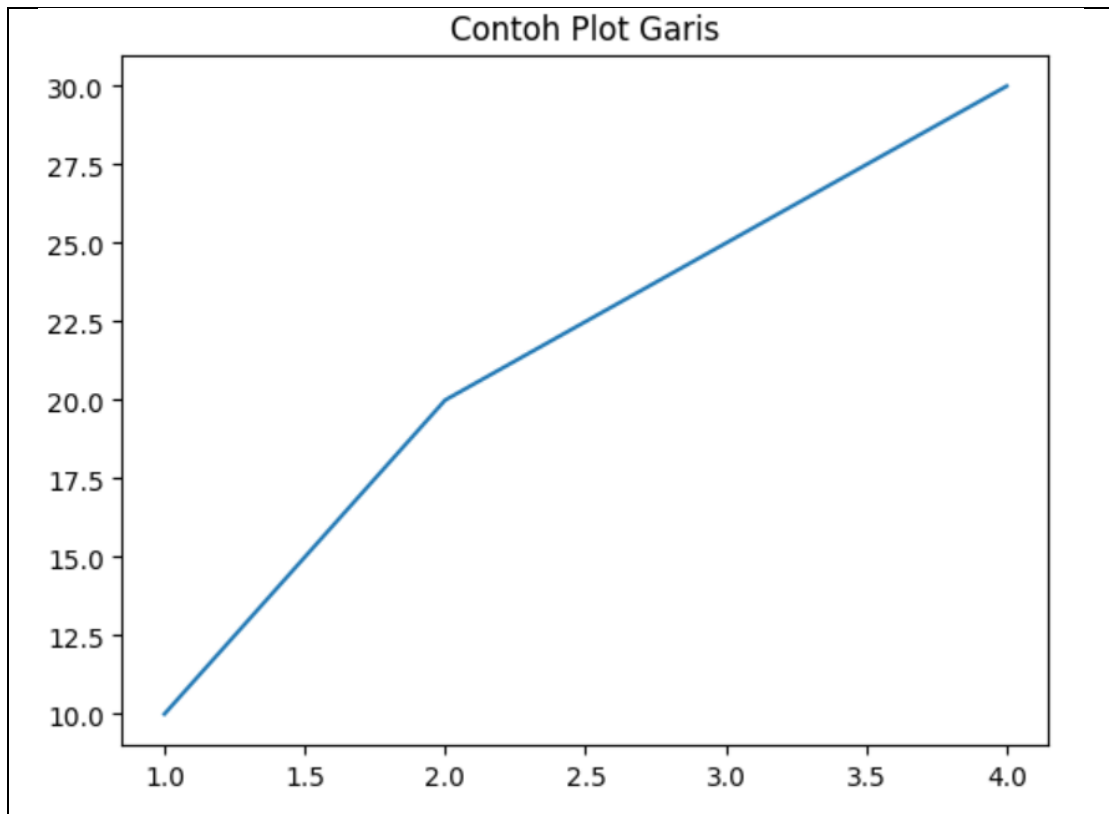
	Nama	Nilai
0	Ali	85
1	Budi	90

1.5. Matplotlib dan Seaborn: Pengantar Awal Visualisasi

Matplotlib dan Seaborn digunakan sebagai Pustaka untuk visualisasi dari data. Berikut Adalah contoh dari penggunaan Matplotlib dan Seaborn.

```
import matplotlib.pyplot as plt  
x = [1, 2, 3, 4]  
y = [10, 20, 25, 30]  
plt.plot(x, y)  
plt.title('Contoh Plot Garis')  
plt.show()
```

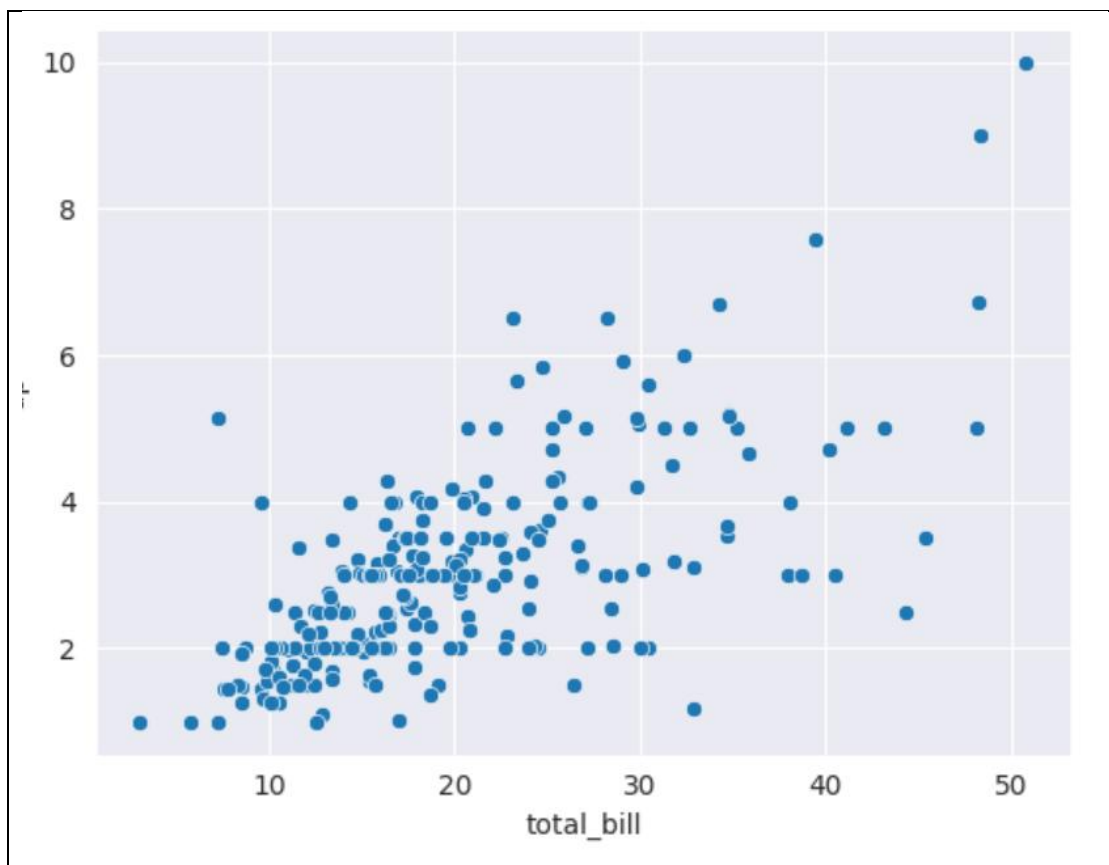
Output



Contoh Penggunaan Seaborn

```
import seaborn as sns
sns.set_style('darkgrid')
tips = sns.load_dataset('tips')
sns.scatterplot(data=tips, x='total_bill', y='tip')
plt.show()
```

Output



1.6. Memuat dan Memahami Dataset

Format CSV (Comma-Separated Values) adalah format berkas teks sederhana yang merepresentasikan data tabular, di mana setiap baris mewakili satu baris data dan setiap nilai dalam baris tersebut dipisahkan oleh tanda koma (atau terkadang titik koma). Untuk memuat berkas CSV ke dalam struktur DataFrame pandas, digunakan fungsi `pd.read_csv()`.

```
import pandas as pd  
df = pd.read_csv('retail_sales.csv')
```

Contoh jika menggunakan Google Collab

```
from google.colab import files  
uploaded = files.upload()
```

Untuk melihat sekilas data yang dimuat pada file csv dapat digunakan perintah berikut

```
df.head()  
df.tail()  
df.shape  
df.columns
```

1.7. Inspeksi Data Mendalam

1.7.1 Informasi Struktur Dataset dengan df.info()

Metode df.info() merupakan salah satu perintah paling penting dalam tahap inspeksi awal, karena dalam satu pemanggilan fungsi tunggal, metode ini menyajikan ringkasan komprehensif mengenai struktur dataset.

```
df.info()
```

Contoh output

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 5000 entries, 0 to 4999  
Data columns (total 8 columns):  
InvoiceNo      5000 non-null object  
StockCode      5000 non-null object  
Description    4990 non-null object  
Quantity       5000 non-null int64  
InvoiceDate    5000 non-null object  
UnitPrice      5000 non-null float64  
CustomerID     4000 non-null float64  
Country        5000 non-null object  
dtypes: float64(2), int64(1), object(5)  
memory usage: 312.6+ KB
```

Metode lainnya yang dapat digunakan adalah df.dtypes, yang menampilkan tipe data setiap kolom secara ringkas tanpa informasi tambahan lain seperti jumlah non-null atau penggunaan memori.

1.7.2 Statistik Deskriptif dengan `df.describe()`

Metode `df.describe()` menghitung dan menampilkan delapan ukuran statistik standar untuk setiap kolom bertipe numerik dalam dataset.

```
df.describe()
```

Secara default, `df.describe()` hanya menghitung statistik untuk kolom bertipe numerik. Untuk memperoleh ringkasan statistik pada kolom kategorik (bertipe object), digunakan parameter tambahan `include='object'`.

```
df.describe(include=' object' )
```

Sebagai pelengkap, untuk memeriksa frekuensi kemunculan setiap kategori secara lebih rinci pada satu kolom tertentu, digunakan metode `value_counts()`.

```
df['Country'].value_counts()  
df['Description'].value_counts().head(10)
```

1.7.3 Mengecek dan Mengukur Missing Value

Missing value atau nilai yang hilang adalah kondisi di mana suatu sel data tidak memiliki nilai — direpresentasikan pandas sebagai NaN (Not a Number).

```
missing = df.isnull().sum()  
missing_pct = (df.isnull().sum() / len(df)) * 100  
missing_df = pd.DataFrame({'Missing': missing, 'Percent':  
missing_pct})  
missing_df[missing_df['Missing'] > 0]
```

Kode tersebut bekerja melalui beberapa tahap. Pertama, `df.isnull()` menghasilkan sebuah DataFrame baru berukuran identik dengan `df`, namun setiap selnya berisi nilai Boolean.

1.8. Penanganan Missing Value Sederhana

Penanganan missing value harus didasarkan pada pemahaman konteks data, besaran persentase nilai yang hilang, dan konsekuensi analitis dari setiap pilihan. Subbab ini membahas dua pendekatan dasar: penghapusan (deletion) dan pengisian (imputation).

1.8.1 Menghapus Baris atau Kolom yang Mengandung Missing Value

Pendekatan pertama adalah menghapus data yang mengandung nilai kosong, baik pada level baris maupun kolom. Pandas menyediakan metode `dropna()` untuk keperluan ini, dengan beberapa variasi penggunaan sesuai kebutuhan.

```
df_clean = df.dropna(subset=['CustomerID'])  
df_clean.shape
```

Selain menghapus berdasarkan kolom tertentu, terdapat variasi lain dari `dropna()` yang perlu dipahami. Untuk menghapus kolom yang memiliki proporsi missing value sangat tinggi (misalnya lebih dari 50% data kosong, sehingga kolom tersebut dianggap tidak lagi informatif), digunakan metode `drop()` dengan menyebutkan nama kolom secara eksplisit.

```
df = df.drop(columns=['NamaKolom'])
```

Sementara itu, memanggil `df.dropna()` tanpa parameter apa pun akan menghapus setiap baris yang memiliki setidaknya satu nilai kosong pada kolom mana pun.

```
df.dropna
```

1.8.2 Mengisi Missing Value melalui Imputasi

Pendekatan kedua, yang menjadi alternatif dari penghapusan, adalah mengisi (impute) nilai yang kosong dengan suatu nilai pengganti yang masuk akal, sehingga baris data tersebut tetap dapat dipertahankan untuk analisis. strategi paling sederhana adalah mengisi nilai kosong tersebut dengan sebuah

konstanta teks yang secara eksplisit menandakan bahwa data memang tidak tersedia.

```
df['Description'] = df['Description'].fillna('Unknown')
```

Untuk kolom numerik, strategi yang lebih umum adalah mengisi nilai kosong dengan ukuran pemusatan data (central tendency) seperti median atau rata-rata, dengan asumsi bahwa nilai yang hilang kemungkinan besar tidak jauh berbeda dari pola umum data yang ada.

```
median_price = df['UnitPrice'].median()  
df['UnitPrice'] = df['UnitPrice'].fillna(median_price)
```

Untuk data yang bersifat runtut waktu (time series), seperti data sensor pabrik yang direkam setiap menit atau setiap jam, terdapat strategi imputasi khusus yang memanfaatkan urutan waktu tersebut, yaitu forward fill (mengisi nilai kosong dengan nilai valid terakhir sebelumnya) dan backward fill (mengisi dengan nilai valid berikutnya).

```
df.fillna(method='ffill')
```

1.9. Visualisasi Data Dasar untuk Eksplorasi

Visualisasi data adalah komponen inti dari EDA yang memungkinkan seorang analis “melihat” pola, sebaran, dan hubungan dalam data secara jauh lebih intuitif dibandingkan hanya membaca angka-angka statistik.

1.9.1 Histogram: Memvisualisasikan Distribusi Data

Histogram adalah jenis grafik yang menunjukkan distribusi frekuensi suatu variabel numerik dengan cara membagi rentang nilai variabel tersebut menjadi sejumlah interval (disebut bin), lalu menghitung dan menampilkan berapa banyak data yang jatuh pada masing-masing interval dalam bentuk batang vertikal.

```
plt.figure(figsize=(10, 5))  
df_clean['Quantity'].hist(bins=50, color='steelblue',  
edgecolor='black')
```

```
plt.title('Distribusi Quantity Pembelian')
plt.xlabel('Quantity')
plt.ylabel('Frekuensi')
plt.show()
```

1.9.2 Boxplot: Ringkasan Lima Serangkai dan Deteksi Outlier

Boxplot (disebut juga box-and-whisker plot atau diagram kotak-garis) adalah visualisasi yang meringkas sebaran data melalui lima nilai statistik utama sekaligus: nilai minimum, kuartil pertama (Q1), median (kuartil kedua), kuartil ketiga (Q3), dan nilai maksimum — dikenal sebagai ringkasan lima serangkai (five-number summary).

```
plt.figure(figsize=(8, 5))
sns.boxplot(x=df_clean['UnitPrice'])
plt.title('Boxplot Harga Satuan')
plt.show()
```

Persamaan untuk menentukan batas outlier berdasarkan IQR dirumuskan sebagai berikut.

```
Q1 = df_clean['UnitPrice'].quantile(0.25)
Q3 = df_clean['UnitPrice'].quantile(0.75)
IQR = Q3 - Q1
low = Q1 - 1.5 * IQR
high = Q3 + 1.5 * IQR
df_filtered = df_clean[(df_clean['UnitPrice'] >= low) &
(df_clean['UnitPrice'] <= high)]
print("Shape sebelum filtering:", df_clean.shape)
print("Shape sesudah filtering:", df_filtered.shape)
```

1.9.3 Scatter Plot: Mengungkap Hubungan Antar-Variabel

Scatter plot (diagram pencar) adalah visualisasi yang menampilkan hubungan antara dua variabel numerik dengan cara memplot setiap pasangan nilai sebagai satu titik pada bidang dua dimensi, di mana satu variabel menempati sumbu horizontal dan variabel lainnya menempati sumbu vertikal.

```
plt.figure(figsize=(8, 6))
plt.scatter(df_clean['Quantity'], df_clean['UnitPrice'],
alpha=0.3)
plt.xlabel('Quantity')
plt.ylabel('Unit Price')
plt.title('Quantity vs UnitPrice')
plt.show()
```

1.9.4 Heatmap Korelasi: Mengukur Kekuatan Hubungan Antar-Variabel Secara Kuantitatif

Heatmap korelasi memberikan ukuran kuantitatif atas kekuatan dan arah hubungan linear antara dua variabel numerik, dinyatakan dalam sebuah angka yang disebut koefisien korelasi.

```
corr = df_clean.corr(numeric_only=True)
plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, cmap='coolwarm', vmin=-1,
vmax=1) plt.title('Heatmap Korelasi')
plt.show()
```